

Absolutely. Here is **Set 8 — 60 more scenario-based GenAI interview questions**, with answers kept in a style you can actually speak in an interview.

■ GenAI 4+ Years Interview — Set 8

361. Your RAG application works in development but fails in production. What do you check?

Answer:

"I would compare the environments first: environment variables, API keys, model configuration, vector database connection, network access, IAM permissions, package versions, and resource limits. I would check logs and traces to identify the exact failing component."

362. The application works after restart but fails again after several hours. What could be happening?

Answer:

"I would check memory leaks, connection pool exhaustion, thread or worker exhaustion, cache growth, queue backlog, and resource utilization over time."

363. Your application's memory keeps increasing. How do you troubleshoot?

Answer:

"I would monitor memory over time and identify which process or component is growing. I would inspect large documents, unbounded caches, conversation history, vector operations, and objects that aren't being released."

364. CPU suddenly reaches 100% during document ingestion. What do you do?

Answer:

"I would identify which ingestion stage is consuming CPU. I could reduce concurrency, batch the workload, move processing to scalable workers, and auto-scale the ingestion layer."

365. Your ingestion workers are overloaded but the API is healthy. What architecture change would help?

Answer:

"I would decouple ingestion from the API using a message queue. The API only accepts the document and creates a job, while workers process the document asynchronously."

■ Async Processing

366. Why shouldn't a PDF upload API generate embeddings synchronously?

Answer:

"Large documents can take a long time to process. A synchronous request can timeout and consume API resources. I would return a job ID and process the document asynchronously."

367. What would the user see during asynchronous ingestion?

Answer:

```
Upload PDF
  ↓
Job Created
  ↓
Processing
  ↓
Extracting
  ↓
Chunking
  ↓
Embedding
  ↓
Indexed
  ↓
Completed
```

"The user can check the job status instead of keeping the HTTP request open."

368. What if the same ingestion job is submitted twice?

Answer:

"I would use an idempotency key or document hash so the system can detect duplicate jobs and avoid duplicate processing."

369. What if an embedding worker fails halfway through?

Answer:

"I would persist job status and process only the failed or incomplete portion if possible. The job can be retried, and persistent failures can go to a dead-letter queue."

■ API Reliability

370. What is a circuit breaker?

Answer:

"A circuit breaker prevents an application from repeatedly calling a failing downstream service. After failures reach a threshold, the circuit opens and requests fail fast until the service recovers."

371. Why use a circuit breaker with an LLM API?

Answer:

"If the LLM provider is unavailable, continuously sending requests can increase latency and overload our application. A circuit breaker protects our system and allows a fallback or graceful error."

372. What is graceful degradation?

Answer:

"Instead of completely failing when one component is unavailable, the system provides a limited but useful response."

Example:

"If the LLM is unavailable, show relevant source documents or a controlled service-unavailable message instead of hallucinating."

373. What is a fallback model?

Answer:

"A fallback model is a secondary LLM used when the primary model is unavailable or exceeds certain limits."

■ Prompt Versioning

374. Why should prompts be versioned?

Answer:

"A prompt change can change application behavior significantly. Versioning allows us to compare prompts, perform testing, audit changes, and roll back to a previous working version."

375. Developer changes the prompt directly in production. Is that good practice?

Answer:

"No. Prompts are part of application behavior and should go through version control, testing, approval, and deployment processes."

376. How would you compare two prompts?

Answer:

"I would run both prompts against the same evaluation dataset and compare correctness, groundedness, relevance, latency, token usage, and cost."

■ Model Versioning

377. Why version the LLM model?

Answer:

"A model update can change output quality, latency, token behavior, or safety behavior. Versioning allows us to reproduce results and roll back if necessary."

378. What happens if the LLM provider retires your model?

Answer:

"I would have a migration plan. I would test the replacement model against our evaluation dataset, compare cost and latency, update the application configuration, and deploy gradually."

379. How do you safely migrate from one LLM to another?

Answer:

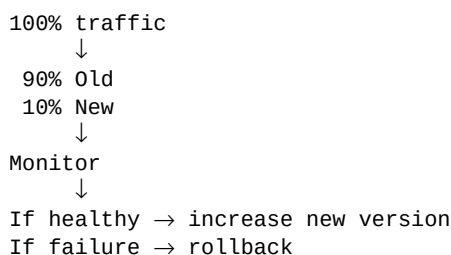
"I would benchmark the new model offline, run A/B or canary traffic, monitor quality and latency, and keep the previous model available for rollback."

■ Canary Deployment

380. What is a canary deployment?

Answer:

"A canary deployment sends a small percentage of production traffic to the new version before fully deploying it."



381. Why is canary deployment useful for GenAI?

Answer:

"GenAI changes can affect quality in unexpected ways. Canary deployment allows us to detect quality, latency, cost, or error problems before exposing the change to all users."

■ Evaluation Scenarios

382. Human reviewers disagree about whether an answer is correct. What do you do?

Answer:

"I would define clear evaluation criteria and guidelines, use multiple reviewers for ambiguous cases, and calculate agreement. For critical applications, I would maintain a curated golden dataset."

383. Why is human evaluation still needed?

Answer:

"Automated metrics are useful, but some aspects such as usefulness, nuance, and factual correctness can be difficult to measure automatically. Human evaluation provides an additional quality signal."

384. Can you rely completely on an LLM to evaluate another LLM?

Answer:

"No. LLM-as-a-judge can be useful, but it can have biases and inconsistencies. I would combine automated evaluation with deterministic checks and human evaluation for important cases."

385. How do you build a golden dataset?

Answer:

"I would collect representative production-style questions, expected answers, relevant source documents, and edge cases. Domain experts should validate the expected answers."

■ Hallucination Scenarios

386. The retrieved context says "5 years" but the LLM says "10 years." What do you call this?

Answer:

"This is a groundedness or faithfulness failure because the generated answer contradicts the retrieved source."

387. How would you automatically detect this?

Answer:

"I could use answer-to-context entailment or an evaluation model, combined with deterministic checks for important values such as dates, numbers, IDs, and policy terms."

388. Why are numbers particularly important to validate?

Answer:

"A small numerical error can significantly change the meaning of an answer. For financial, medical, or policy applications, numerical values should receive additional validation."

■ Structured Data + LLM

389. Your company has both PDFs and SQL databases. Should everything go into a vector database?

Answer:

"No. Unstructured documents are suitable for RAG, while structured transactional data should generally remain in SQL or another authoritative database. I would use the appropriate source for each type of information."

390. User asks:

"How many customers purchased product X last month?"

Answer:

"I would query the database or analytics system rather than rely on RAG."

391. User asks:

"What is the company's return policy for product X?"

Answer:

"If the policy is stored in documents, I would use RAG."

392. User asks:

"How many customers purchased product X, and what is the return policy?"

Answer:

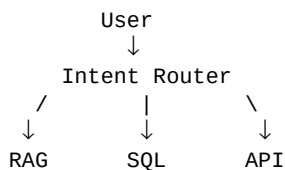
"I would retrieve the transaction count from the database and the policy from the document store, then combine the results in the response."

■ Agent Routing

393. What is an intent router?

Answer:

"An intent router determines what type of request the user has and sends it to the appropriate workflow."



394. Why use routing?

Answer:

"It prevents every request from going through the same expensive workflow and ensures each question uses the appropriate data source."

395. What if the router selects the wrong workflow?

Answer:

"I would monitor routing accuracy, add confidence thresholds, improve the classification logic, and provide a fallback path for uncertain requests."

■ Security

396. Where should API keys be stored?

Answer:

"API keys should not be hardcoded in source code. They should be stored in a secure secrets-management system or protected environment configuration."

397. A developer accidentally commits an LLM API key to Git. What do you do?

Answer:

"I would immediately revoke or rotate the exposed key, remove it from active use, check whether it was used, and update the secret-management process. Removing it from Git alone isn't enough because the key may already be compromised."

398. Why should secrets not be placed in prompts?

Answer:

"Prompts and model interactions can be logged, monitored, or exposed to unintended components. Secrets should be managed outside the LLM context."

399. Can the LLM be trusted with database passwords?

Answer:

"No. Credentials should never be exposed to the model. The application should manage credentials and provide the LLM only with controlled tools or results."

■ Multi-Tenant RAG

400. What is multi-tenant RAG?

Answer:

"A RAG system serving multiple organizations or customers while keeping their data logically isolated."

401. How do you prevent Tenant A from retrieving Tenant B's documents?

Answer:

"Every document and chunk would have a tenant ID, and every retrieval request would enforce the authenticated tenant ID as a mandatory filter."

402. What if a developer forgets to apply the tenant filter?

Answer:

"I would enforce tenant isolation at a lower data-access layer rather than relying only on individual application code paths. I would also add automated security tests specifically for cross-tenant access."

■ Cache Security

403. User A asks a question and the answer is cached. User B asks the same question. Can User B receive it?

Answer:

"Only if the answer is not user- or tenant-specific and access controls permit it. Otherwise, the cache key must include the appropriate tenant or user context."

404. What information should be included in a secure cache key?

Answer:

Potentially:

```
tenant_id
user/role scope
query
model version
prompt version
document/index version
```

"The exact key depends on whether the response is user-specific."

■ Data Lifecycle

405. A document is deleted. What happens to its embeddings?

Answer:

"The corresponding vectors and metadata should also be removed or marked inactive so the deleted document cannot continue appearing in retrieval."

406. A document is marked inactive but its vector remains. Is that a problem?

Answer:

"Yes, unless retrieval filters inactive documents. I would either delete the vector or ensure the active-status filter is mandatory."

407. How do you handle document deletion safely?

Answer:

"I would identify all chunks belonging to the document using document ID and remove or deactivate them. I would then validate that retrieval no longer returns them."

■ Cost Optimization

408. What is the biggest cost driver in a RAG system?

Answer:

"It depends on the architecture, but LLM token usage can be a major cost driver. Embedding generation, reranking, infrastructure, and storage can also contribute."

409. How do you reduce embedding cost?

Answer:

"Avoid re-embedding unchanged documents, use content hashes, batch embedding requests, process only changed documents, and select an appropriate embedding model."

410. How do you reduce retrieval cost?

Answer:

"Use efficient indexing, metadata filtering, appropriate Top-K, and avoid unnecessarily large candidate sets."

■ Performance

411. Why use batching?

Answer:

"Batching allows multiple documents or texts to be processed together, reducing overhead and improving throughput."

412. When should you avoid excessive batching?

Answer:

"Very large batches can increase memory usage, latency, or API limits. I would choose batch sizes based on model and infrastructure constraints."

413. What is throughput?

Answer:

"Throughput is the amount of work processed per unit of time, such as documents per minute or requests per second."

414. What is latency?

Answer:

"Latency is the time taken to complete an individual request."

■ Architecture

415. Why separate ingestion and query services?

Answer:

"They have different workloads. Ingestion is CPU/GPU-intensive and asynchronous, while querying needs low latency. Separating them allows independent scaling and prevents ingestion from affecting users."

416. Why use object storage for original PDFs?

Answer:

"Object storage is designed for scalable, durable storage of files and is usually more appropriate than storing large PDF binaries directly in the vector database."

417. What should the vector database store?

Answer:

"It should store embeddings, chunk references, and metadata required for retrieval. The original document can remain in object storage."

418. Why keep chunk text with the vector?

Answer:

"It allows the retrieval layer to return the relevant content directly for prompt construction. Alternatively, the vector can reference the original chunk stored elsewhere."

■ Advanced Scenario

419. User asks a question that requires information from 20 pages of one document. What do you do?

Answer:

"I wouldn't necessarily send all 20 pages directly to the LLM. I would retrieve relevant chunks, possibly retrieve neighboring chunks or use hierarchical retrieval, then summarize or synthesize the required information."

420. Your RAG retrieves the right document but wrong section. What do you investigate?

Answer:

"I would inspect chunking granularity, document structure, metadata, embedding quality, and query formulation. Hierarchical or section-aware retrieval may improve this."

■ 421. Interviewer: "Why did you choose RAG instead of fine-tuning?"

Strong answer:

"I chose RAG because our knowledge base changes frequently. With RAG, when a document changes, I can process the new document, generate embeddings, and update the vector store without retraining the LLM. Fine-tuning is more suitable when I need to change the model's behavior, style, or task-specific capabilities rather than continuously update factual knowledge."

■ 422. Interviewer: "Your RAG gives irrelevant answers. Give me your troubleshooting process."

Answer:

"First I would inspect the retrieved chunks and similarity scores. If retrieval is poor, I would check chunk size, overlap, embeddings, Top-K, similarity threshold, metadata filtering, and reranking. If retrieval is correct but the answer is wrong, I would check the prompt, context construction, and model behavior. Finally, I would validate the fix using a retrieval and answer evaluation dataset."

■ 423. Interviewer: "Your RAG answers are too long."

Answer:

"Since retrieval is already good, I would focus on generation. I would update the prompt to request concise answers, define a response format, limit output tokens, and remove unnecessary context. Then I would compare answer relevance and length before and after the change."

■ 424. Interviewer: "Your RAG doesn't know when to say 'I don't know'."

Answer:

"I would introduce a retrieval confidence threshold. If no retrieved chunk exceeds the threshold, the generation step should not proceed normally. The prompt should also explicitly instruct the model to say 'I don't know based on the available documents' when the context doesn't support an answer."

■ 425. Interviewer: "How do you scale your GenAI application?"

Answer:

"I would put the application behind a load balancer and horizontally scale stateless API instances. For document ingestion, I would use queues and auto-scaled workers. I would also use caching, rate limiting, monitoring, and independent scaling for the vector database and ingestion pipeline."

■ 426. Interviewer: "How do you handle a sudden traffic spike?"

Answer:

"I would use auto-scaling and load balancing for the application layer, queue asynchronous workloads, apply rate limits, use caching where possible, and monitor downstream LLM rate limits. I would make sure the system fails gracefully rather than allowing the entire application to become unavailable."

■ 427. Interviewer: "What if the LLM API becomes slow?"

Answer:

"I would first verify whether the latency is from our application or the provider. I would use timeouts, limited retries with exponential backoff, monitoring, and possibly a fallback model. I would avoid allowing slow downstream calls to exhaust application resources."

■ 428. Interviewer: "How do you control GenAI cost?"

Answer:

"I would monitor token usage and cost per request, reduce unnecessary context, optimize Top-K, limit output tokens, cache repeated requests, batch embeddings, avoid reprocessing unchanged documents, and use smaller models for simpler tasks."

■ 429. Interviewer: "How do you make RAG secure?"

Answer:

"I would enforce authentication and authorization before retrieval, isolate tenants, filter documents using access metadata, protect API keys, sanitize inputs, protect against prompt injection, avoid exposing sensitive logs, and validate outputs."

■ 430. "What makes you a 4+ years GenAI engineer rather than someone who just knows RAG?"

Strong answer:

"I focus not only on building the RAG pipeline but also on making it production-ready. I consider retrieval quality, prompt engineering, evaluation, security, scalability, observability, latency, cost, document versioning, failure handling, and continuous improvement. I troubleshoot problems by identifying the failing component and validating the fix with measurable metrics."

■ Your core interview flow

For almost every scenario, think:

1. Identify → 2. Measure → 3. Troubleshoot → 4. Fix → 5. Validate → 6. Monitor

Example:

"First I identify whether the issue is retrieval or generation. Then I check metrics, logs, and retrieved chunks. I apply the appropriate fix, validate it against an evaluation dataset, and monitor the production impact."